

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №3

Выполнила:

Ширкунова Мария

J3211

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Мигрировать ранее разработанное приложение (в рамках ЛР1 и ЛР2) на фреймворк Vue.js. Приложение должно соответствовать следующим требованиям:

- Должен быть подключён роутер;
- Должна быть реализована работа с внешним API (желательно посредством axios);
- Разумное деление на компоненты (продемонстрировать понимание компонентного подхода);
- Использование composable для выделения повторяющегося функционала в отдельные файлы.

В качестве предметной области выступает сервис для управления личными финансами и бюджетом, разработанный в ЛР1 и ЛР2.

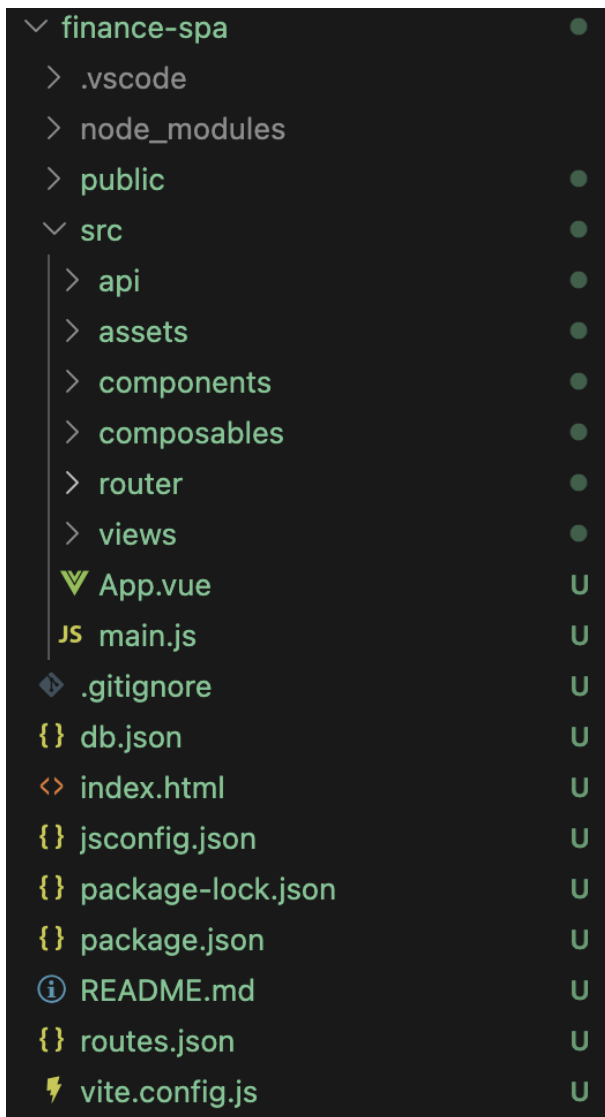
Ход работы

1. Инициализация проекта

Проект создан с помощью официального шаблона Vue с использованием менеджера зависимостей npm.

При инициализации проекта подключён официальный пакет **Vue Router**. Дополнительно установлены библиотека **axios** для работы с внешним API и UI-фреймворк **Bootstrap 5** для сохранения визуальной составляющей, разработанной в предыдущих лабораторных работах.

Для моковой части API сохранены настройки из ЛР2: установлены пакеты json-server и json-server-auth, добавлен npm-скрипт для запуска сервера авторизации на порту 3000.



2. Архитектура приложения

Приложение разделено на следующие логические части:

- **src/api/axios.js** — единый инстанс axios с базовым URL и интерсептором, автоматически добавляющим JWT-токен в заголовок Authorization каждого запроса;
- **src/composables/** — модули с переиспользуемой логикой (useAuth, useApi, useTransactions);
- **src/components/** — переиспользуемые презентационные компоненты (AppNavbar, AccountCard, TransactionRow, FilterPanel, TransactionFormModal);
- **src/views/** — компоненты-страницы, привязанные к маршрутам;

- **src/router/index.js** — конфигурация маршрутизатора с защитой роутов через navigation guard.

```
J3211 > Ширкунова Мария > labs > lab-1 > finance-spa > src > api > JS axios.js > ...
1   import axios from 'axios'
2
3   const api = axios.create({
4     |   baseURL: 'http://localhost:3000'
5   })
6
7   api.interceptors.request.use((config) => {
8     |   const token = localStorage.getItem('accessToken')
9     |   if (token) {
10      |     config.headers.Authorization = `Bearer ${token}`
11      |   }
12      |   return config
13    })
14
15   export default api
```

3. Маршрутизация на клиентской части (Vue Router)

Подключение Vue Router выполнено в режиме createWebHistory (History API). Для каждого маршрута заданы meta-поля: requiresAuth для защищённых страниц и guestOnly для страниц входа и регистрации.

Защита маршрутов реализована через глобальный хук router.beforeEach: при отсутствии токена в localStorage пользователь автоматически перенаправляется на страницу входа; авторизованный пользователь не может попасть на страницы /login и /register.

Навигация между страницами осуществляется через компонент <router-link>, что обеспечивает работу приложения без перезагрузки страницы — это и есть ключевое отличие SPA от MPA.

```

J3211 > Ширкунова Мария > labs > lab-1 > finance-spa > src > router > JS index.js > routes > component
1  import { createRouter, createWebHistory } from 'vue-router'
2
3  import LoginView from '@views/LoginView.vue'
4  import RegisterView from '@views/RegisterView.vue'
5  import DashboardView from '@views/DashboardView.vue'
6  import TransactionsView from '@views/TransactionsView.vue'
7  import ReportsView from '@views/ReportsView.vue'
8  import IntegrationsView from '@views/IntegrationsView.vue'
9
10 const routes = [
11   { path: '/login', name: 'login', component: LoginView, meta: { guestOnly:
12   { path: '/register', name: 'register', component: RegisterView, meta: { guestOnly:
13   { path: '/', name: 'dashboard', component: DashboardView, meta: { requiresAut
14   { path: '/transactions', name: 'transactions', component: TransactionsView, meta: { requiresAut
15   { path: '/reports', name: 'reports', component: ReportsView, meta: { requiresAut
16   { path: '/integrations', name: 'integrations', component: IntegrationsView, meta: { requiresAut
17 ]
18
19 const router = createRouter({
20   history: createWebHistory(),
21   routes
22 })
23
24 router.beforeEach((to) => {
25   const isAuth = !!localStorage.getItem('accessToken')
26   if (to.meta.requiresAuth && !isAuth) return { name: 'login' }
27   if (to.meta.guestOnly && isAuth) return { name: 'dashboard' }
28 })
29
30 export default router

```

4. Работа с внешним API через axios

Вместо разрозненных вызовов fetch из ЛР2 создан единый axios-инстанс. Интерсептор запросов автоматически подставляет токен авторизации, что избавило от дублирования кода в каждом методе.

В composable useApi инкапсулированы реактивные состояния loading и error, благодаря чему компоненты могут отображать индикатор загрузки и обрабатывать ошибки декларативно через директивы v-if.

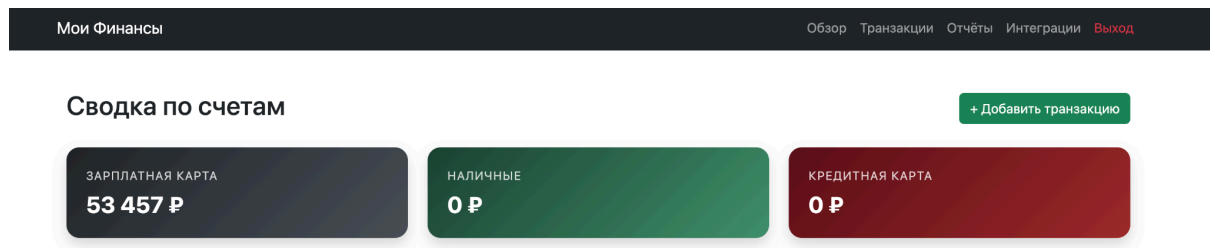
Name	X	Headers	Payload	Preview	Response	Initiator	Timing
 transactions?userId=4	Access-Control-Allow-Credentials				true		
	Access-Control-Allow-Origin				http://localhost:5174		
	Cache-Control				no-cache		
	Connection				keep-alive		
	Date				Sat, 30 May 2026 12:23:26 GMT		
	Etag				W/"1ca-NcXPo86iR1V4Cw3Pz6iWRC3SHSM"		
	Expires				-1		
	Keep-Alive				timeout=5		
	Pragma				no-cache		
	Vary				Origin		
	X-Content-Type-Options				nosniff		
	X-Powered-By				Express		
	▼ Request Headers ☐ Raw						
Accept					application/json, text/plain, */*		
Accept-Encoding					gzip, deflate, br, zstd		
Accept-Language					ru,en;q=0.9		
Authorization					Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6InVzZXI1NUBnbWVpY20iLCJpYXQiOiJlE3ODAxNDIzMyIsImV4cCI6MTc0MDE0NjMyMSwic3VlIjoibWRC3SHSM6ouSUMrPdzPFRClJB0tPQ5ICI		
Connection					keep-alive		
Host					localhost:3000		
If-None-Match					W/"1ca-NcXPo86iR1V4Cw3Pz6iWRC3SHSM"		
Origin					http://localhost:5174		
Referer					http://localhost:5174/		

5. Компонентный подход

Приложение разделено на компоненты по принципу единственной ответственности:

- **AppNavbar.vue** — общая навигационная панель, использует `useAuth` для кнопки выхода;
- **AccountCard.vue** — карточка счёта, принимает props (`name`, `type`, `balance`);
- **TransactionRow.vue** — строка таблицы транзакции с цветовым выделением расходов/доходов через `computed`;
- **FilterPanel.vue** — панель фильтров, реализована через `v-model` (паттерн `modelValue` + `update:modelValue`) — двусторонняя связь с родителем;
- **TransactionFormModal.vue** — модальное окно добавления транзакции, эмитит событие `submit` с данными формы.

Компоненты переиспользуются на разных страницах (AppNavbar — на всех защищённых view, AccountCard — в цикле v-for на дашборде).



6. Composable для выделения повторяющегося функционала

Реализованы три composable-модуля:

- **useAuth.js** — управление авторизацией. Состояния user и token объявлены вне функции — это паттерн «синглтон», за счёт чего любая часть приложения работает с одним и тем же состоянием пользователя. Метод login сохраняет токен, обновляет реактивное состояние и выполняет программный редирект через router.push;
- **useApi.js** — универсальная обёртка над axios, инкапсулирует loading, error и метод request;
- **useTransactions.js** — бизнес-логика работы с транзакциями. Здесь объявлены реактивные источники данных (transactions, filters) и вычисляемые свойства (computed): accounts (балансы счетов, пересчитываемые из транзакций) и filteredTransactions (отфильтрованный список). За счёт реактивности при добавлении новой транзакции автоматически обновляются и таблица, и карточки счетов — без ручных вызовов методов обновления, как это было в ЛР2.

В отличие от миксинов Vue 2, использование composable — явное (через import), что улучшает читаемость и упрощает отладку.

7. Страницы приложения

Все страницы из ЛР2 мигрированы на Vue:

- **LoginView / RegisterView** — формы с реактивными полями через v-model, при успешной авторизации useAuth сохраняет токен и редиректит на дашборд.

- **DashboardView** — основная страница со сводкой по счетам. Карточки счетов формируются автоматически через computed-свойство из загруженных транзакций. Кнопка открывает модальное окно TransactionFormModal; при отправке формы вызывается метод addTransaction, после чего реактивно обновляются и счета, и таблица.
- **TransactionsView** — таблица истории транзакций с панелью фильтров. Связь между FilterPanel и таблицей построена через v-model: изменение полей в фильтре триггерит пересчет computed-свойства filteredTransactions. В отличие от LP2, фильтрация работает мгновенно, без кнопки «Применить».
- **ReportsView** — страница с прогресс-барами использования бюджета.
- **IntegrationsView** — страница имитации подключения банковских аккаунтов.

Вывод

В результате выполнения лабораторной работы приложение «Мои Финансы» успешно мигрировано с многостраничной архитектуры (MPA) на одностраничное приложение (SPA) с использованием фреймворка Vue.js. Все требования к лабораторной работе выполнены:

- Подключён **Vue Router** с защитой маршрутов через meta-поля и beforeEach navigation guard;
- Реализована работа с внешним API через **axios**: создан единый инстанс с интерсептором для автоматической подстановки JWT-токена;
- Приложение разделено на **разумные переиспользуемые компоненты** (AppNavbar, AccountCard, TransactionRow, FilterPanel, TransactionFormModal) и страницы-views, что демонстрирует понимание компонентного подхода;
- Повторяющаяся логика вынесена в **composables** (useAuth, useApi, useTransactions), что устранило дублирование кода и обеспечило централизованное управление состоянием.

В ходе работы на практике закреплены материалы лекций:

- **Vue.js: основы** — освоен синтаксис Composition API (<script setup>), реактивные ссылки ref, реактивные объекты reactive, вычисляемые свойства computed;
- **Компонентный подход** — отработана работа с props, emits, паттерн v-model для пользовательских компонентов;
- **Vue Router** — изучена клиентская маршрутизация на History API, реализованы navigation guards и meta-поля для разграничения публичных и приватных страниц;
- **Composables** — изучена концепция переиспользуемой логики, в том числе паттерн «синглтон» (общее состояние через объявление ref вне функции) для централизованного хранения данных пользователя;
- **Работа с API** — освоена работа с библиотекой axios, в частности — создание единого инстанса и использование интерсепторов для централизованной обработки заголовков авторизации.

Главным практическим преимуществом перехода на Vue.js стало устранение императивных манипуляций DOM (innerHTML, ручной перерисовки таблицы) и переход к декларативной реактивной модели: данные изменяются — UI обновляется автоматически.